



برنامه نویسی شی گرا

Object Oriented Programming

مدرس:

مریم خراشادیزاده

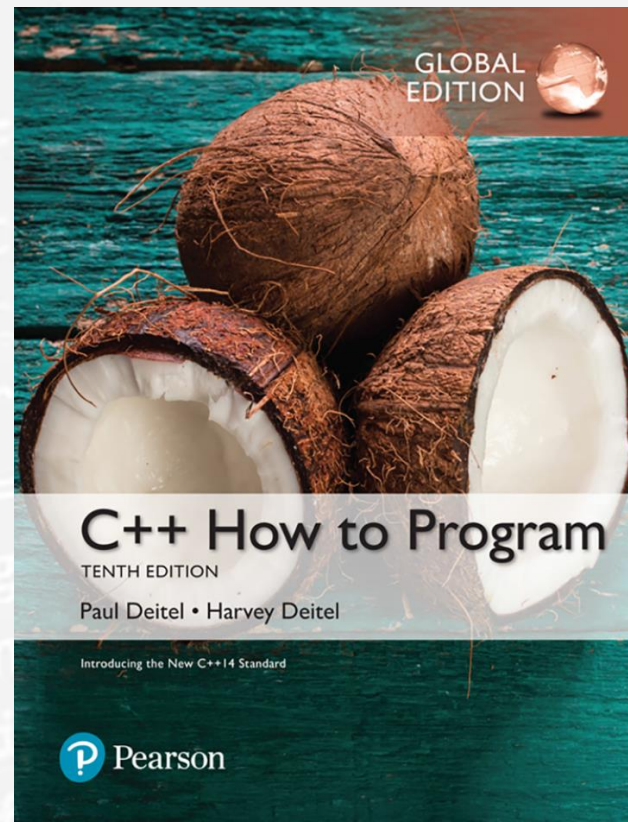


فهرست مطالب

- آشنایی با مفاهیم و ویژگی‌های پایه شی گرای
- تعریف کلاس، سطوح دسترسی اعضای کلاس و تعریف شی
- توابع مقداردهی (Setter) و دستیابی (Getter)
- توابع سازنده (Constructor) و نابودکننده (Destructor)
- توابع و کلاسهای دوست (Friend)
- اشیای ثابت و توابع عضو ثابت (Constant)
- اعضای ایستای کلاس (Static Members)
- ترکیب کلاسها (Composition)
- سربار گذاری عملگرها (Operator Overloading)
- وراثت (Inheritance)
- چند ریختی (Polymorphism)
- فایلها



- **C++ How to Program**
 - Paul Deitel, Harvey Deitel





ارزیابی درس

- آزمون میان ترم: ۳۰٪
- آزمون پایان ترم: ۴۰٪
- تکالیف برنامه نویسی و پروژه: ۳۰٪

- نکته ۱: برای گذراندن درس باید حداقل ۵۰٪ نمره آزمونهای عملی کسب شود.
- نکته ۲: برای گذراندن درس باید حداقل ۵۰٪ نمره تکالیف برنامه نویسی و پروژه کسب شود.



سبک‌های برنامه‌نویسی

• برنامه‌نویسی ساخت‌یافته (Structured Programming):

- در این سبک برنامه‌نویسی، برنامه به صورت مجموعه‌ای از فعالیت‌ها در نظر گرفته می‌شود که باید بر روی داده‌ها اجرا شود. هر مساله پیچیده‌ای به چند مساله کوچکتر تجزیه می‌شود تا مسایل ایجاد شده به راحتی قابل درک و پیاده‌سازی باشند؛ به عبارت دیگر برنامه نویس سعی می‌کند توابعی (Function) بنویسد که نیازمندیهای سیستم را برطرف کند. در نهایت این توابع به هم پیوند می‌خورند تا جریان منطقی برنامه برقرار شود.

• برنامه‌نویسی شی‌گرا (Object Oriented Programming):

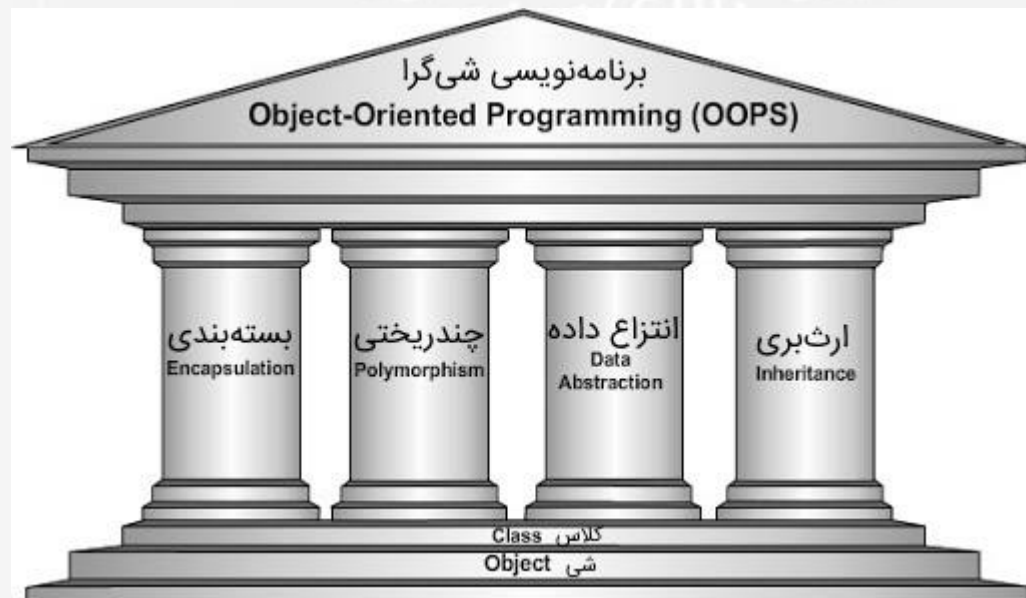
- در این سبک برنامه‌نویسی با الهام از دنیای واقعی، برنامه مجموعه‌ای از موجودیتهایی (Object) در نظر گرفته می‌شود که دارای خصوصیات (Attribute) و رفتارهایی (Behavior) هستند. از کنار هم قرار گرفتن (مونتاژ) این قطعات نرم‌افزاری و تعامل آنها برنامه ایجاد می‌شود. به این ترتیب سرعت تولید نرم‌افزار و همچنین تست، عیب‌یابی و پشتیبانی سریعتر و راحتتر صورت می‌گیرد.



اصول برنامه نویسی شی گرا

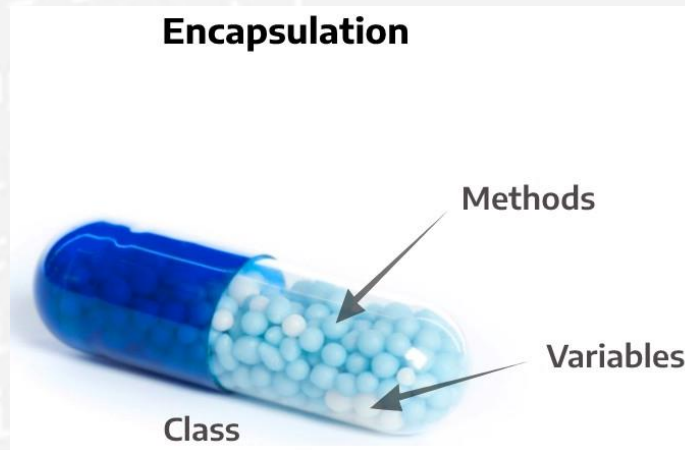
• شی گرایی چهار اصل و خصوصیت کلیدی زیر را شامل می شود:

- بسته بندی (Encapsulation)
- انتزاع (Abstraction)
- وراثت (Inheritance)
- چندریختی (Polymorphism)



بسته‌بندی (Encapsulation)

بسته‌بندی به معنی قرار دادن خصوصیات و رفتارهای (متدها و اعضای داده‌ای) مرتبط به یک موجودیت (شی) در یک بسته است. داده‌های بسته‌بندی شده از دسترس دیگر اشیا مخفی می‌شوند و آن‌ها به این داده‌ها دسترسی نخواهند داشت. به عبارت دیگر متدهای موجود در هر شی فقط به داده‌های همان شی دسترسی دارند و متدهای اشیای دیگر به داده‌های شی مورد نظر دسترسی ندارند. مفهوم بسته‌بندی راهکاری برای «پنهان‌سازی داده» (Data Hiding) نیز شناخته می‌شود.



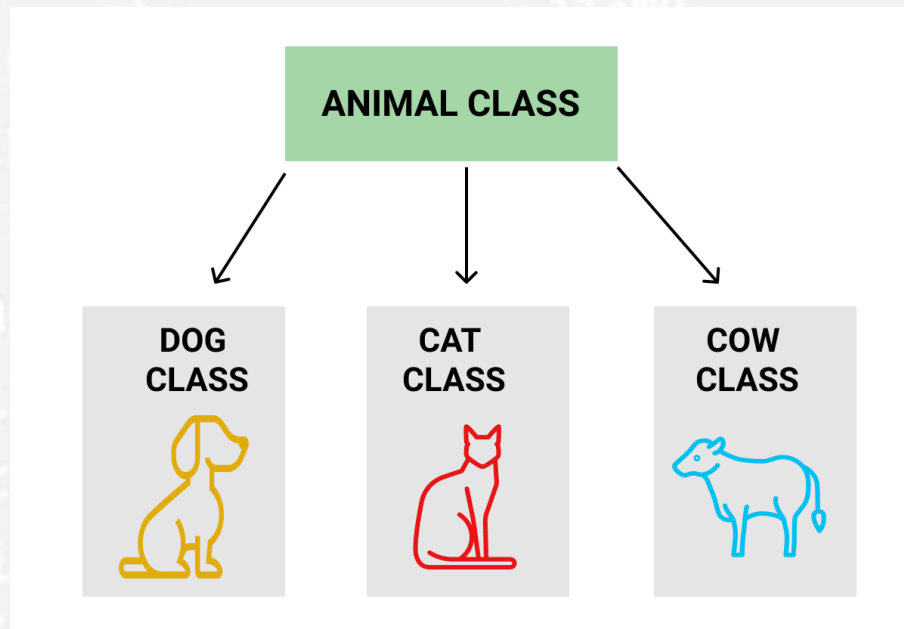
انتزاع (Abstraction)

انتزاع یا تجرید به معنی توانایی بازنمایی (نمایش) داده‌ها در یک سطح کاملاً مفهومی و بدون جزئیات است. به عبارت دیگر انتزاع مخفی کردن پیچیدگیهای پیاده‌سازی داخلی و نشان دادن مجموعه‌ای از خدمات ارائه شده برای استفاده از شی است.



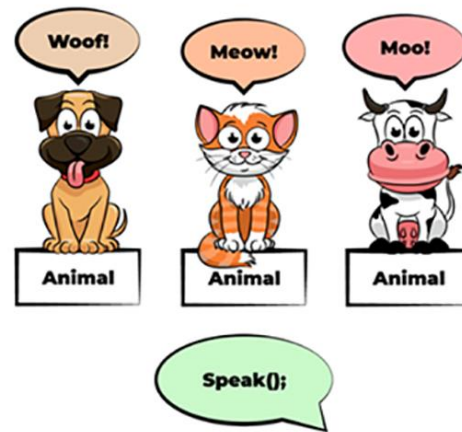
وراثت (Inheritance)

وراثت روشی است که با استفاده از آن یک شی خصوصیت‌های شی دیگری را کسب می‌کند یا به ارث می‌برد. شی جدید می‌تواند خصوصیات و رفتارهای مختص به خود را نیز داشته باشد. وراثت امکان استفاده مجدد از کد و ارتقاء سازمان‌دهی سلسله مراتبی اشیا را فراهم می‌کند.



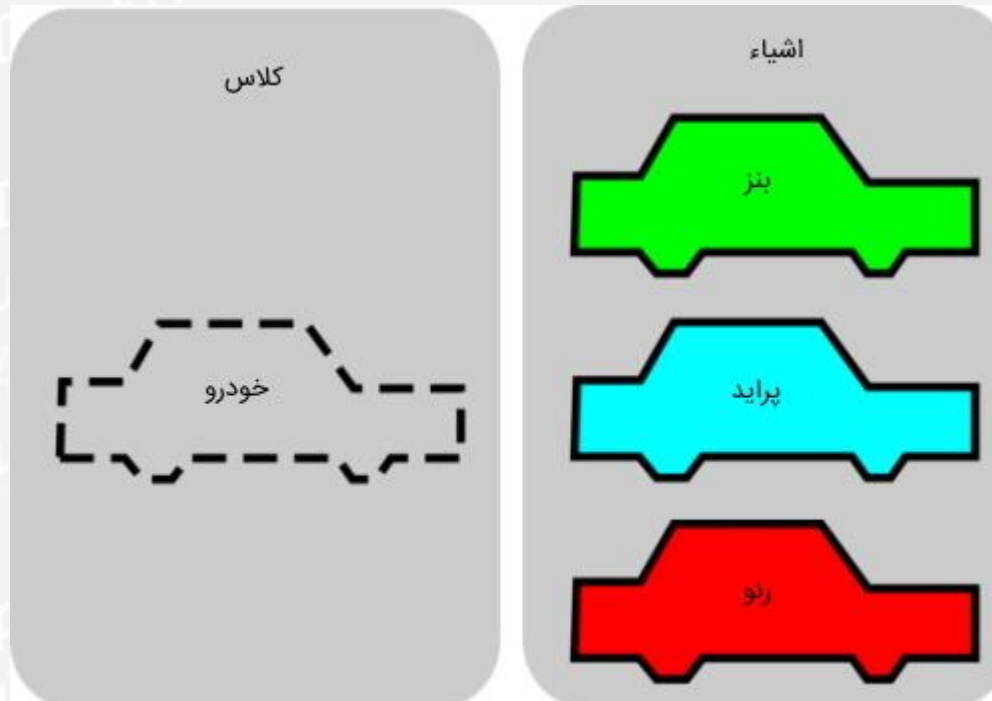
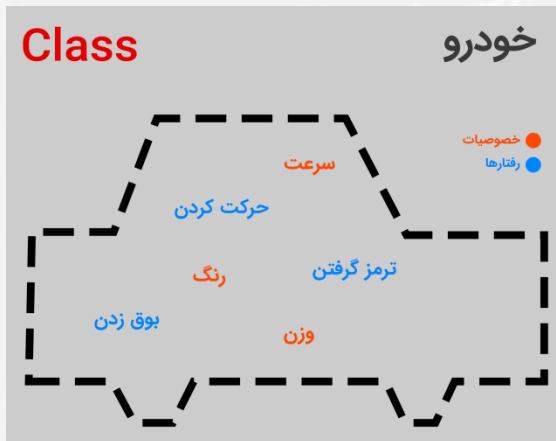
چندریختی (Polymorphism)

چندریختی یا چند شکلی را می‌توان امکان اجرای یک پیام به شکل‌های گوناگون تعریف کرد. به عبارت دیگر می‌توان این مفهوم را «عمومیت بخشی» (Generalization) نیز نامید؛ زیرا یک نام کلی می‌تواند برای همه زیرکلاس‌هایی اعمال شود که از آن مشتق شده‌اند؛ ولی در عمل می‌تواند به شکل‌های گوناگونی (متدهای مختلفی) اجرا شود.



مفهوم کلاس و شی

- **کلاس (class)** عنصر اصلی در برنامه‌نویسی شی‌گرا در زبان برنامه‌نویسی C++ است. کلاس یک نوع داده تعریف شده توسط کاربر است که داده‌ها و توابع عضو مربوط به خودش را نگهداری می‌کند. می‌توان از نوع یک کلاس یک نمونه (Instance) یا شی ایجاد کرد. کلاس درست مثل یک نقشه ساخت برای یک شی است. برای مثال، می‌توان کلاس خودرو را در نظر گرفت و بنز، پراید و رنو را نمونه‌هایی از آن دانست.



ایجاد کلاس در C++

- **مثال:** برنامه‌ای بنویسید که شعاع دایره را دریافت کند و محیط و مساحت دایره‌ای را محاسبه کند.

- مفهوم **دایره** موجودیت قابل شناسایی در این مثال است که ویژگی به نام **شعاع** دارد و می‌توان برای آن دو رفتار **محاسبه محیط** و **محاسبه مساحت** را در نظر گرفت.



تعریف کلاس در C++

- در C++ برای تعریف یک کلاس از کلمه کلیدی «class» استفاده می‌شود.
- بدنه کلاس درون {} قرار داده می‌شود.
- تعریف کلاس به ; ختم می‌شود.
- یک کلاس می‌تواند شامل اعضای داده‌ای (متغیرها) و مجموعه‌ای از متدها (توابع) باشد.
- تعریف کلاس فضایی برای متغیرها رزرو نمی‌کند، بلکه برای استفاده از کلاس باید یک نمونه از کلاس ایجاد کرد.
- هر نمونه (شی) از کلاس کپی مختص به خود را از اعضای داده‌ای کلاس دارد.
- اعضای کلاس می‌توانند یکی از سطوح دسترسی زیر را داشته باشند:
 - عمومی (public): این اعضا بعد از ایجاد یک نمونه از کلاس قابل دسترسی هستند.
 - خصوصی (private): این اعضا فقط درون توابع کلاس (و دوستان کلاس) قابل دسترسی هستند.
 - حفاظت شده (protected): این اعضا درون کلاس و سلسله مراتب ارث‌بری قابل دسترسی هستند (جزئیات در بخش ارث‌بری توضیح داده خواهد شد).
- به صورت پیش‌فرض (در صورت ذکر نکردن سطح دسترسی) اعضای کلاس خصوصی در نظر گرفته می‌شود (اصل مخفی کردن اطلاعات).
- برای دسترسی به هر عضو یک شی (عضو داده‌ای یا تابع عضو) از عملگر . استفاده می‌شود.
- به اعضای عمومی کلاس رابط کلاس گفته می‌شود.



```

1  #include <iostream>
2
3  using namespace std;
4  // تعریف کلاس
5  class Circle
6  {
7      public:
8
9          int Radius;
10
11          // تابع محاسبه محیط
12          float getPerimeter()
13          {
14              return 2 * Radius * 3.14;
15          }
16          // تابع محاسبه مساحت
17          float getArea()
18          {
19              return Radius * Radius * 3.14;
20          }
21      };
22
23
24  int main()
25  {
26      Circle c1, c2, circles[5];
27      c1.Radius = 10;
28      cout << "Perimeter of c1 is " << c1.getPerimeter() << endl;
29      cout << "Area of c1 is " << c1.getArea() << endl;
30
31      c2.Radius = 20;
32      cout << "Perimeter of c2 is " << c2.getPerimeter() << endl;
33      cout << "Area of c2 is " << c2.getArea() << endl;
34      for (int i = 0; i < 5; i++)
35      {
36          cin >> circles[i].Radius;
37          cout << "Perimeter of circels[" << i << "] is " << circles[i].getPerimeter() << endl;
38          cout << "Area of circels[" << i << "] is " << circles[i].getArea() << endl;
39      }
40      return 0;
41  }

```

```

"C:\Users\Maryam\courses\Advanced programming\FirstCircl...
Perimeter of c1 is 62.8
Area of c1 is 314
Perimeter of c2 is 125.6
Area of c2 is 1256
1
Perimeter of circels[0] is 6.28
Area of circels[0] is 3.14
2
Perimeter of circels[1] is 12.56
Area of circels[1] is 12.56
3
Perimeter of circels[2] is 18.84
Area of circels[2] is 28.26
4
Perimeter of circels[3] is 25.12
Area of circels[3] is 50.24
-5
Perimeter of circels[4] is -31.4
Area of circels[4] is 78.5

Process returned 0 (0x0)   execution time : 30.153 s
Press any key to continue.

```



توابع مقداردهی و دستیابی

- معمولا اعضای داده‌ای برای جلوگیری از دستکاریهای نادرست و غیرمجاز **خصوصی** در نظر گرفته می‌شوند.
- مقدار دهی اعضای داده‌ای خصوصی از روش غیرمستقیم و با استفاده از توابع عضو کلاس انجام می‌شود. این توابع معمولا توابع **set** نامیده می‌شوند و معمولا با ترکیب کلمه **set** و نام عضو داده‌ای نامگذاری می‌شوند.
- برای دریافت مقدار داده‌های خصوصی نیز از توابع عضو استفاده می‌شود. این توابع معمولا توابع **get** نامیده می‌شوند و معمولا با ترکیب کلمه **get** و نام عضو داده‌ای نامگذاری می‌شوند.
- توابع **set** و **get** به صورت عمومی معرفی می‌شوند.
- به طور کلی پیاده‌سازی متدهای کلاس مشابه توابع در **C++** است و هر متد کلاس می‌تواند پارامتر ورودی (حتی با مقادیر پیش فرض)، متغیرهای محلی و خروجی داشته باشد.
- بهتر است پیاده‌سازی توابع کلاس (خصوصا توابع طولانی) را خارج از بدنه کلاس انجام داد.
- برای این کار هنگام پیاده‌سازی تابع، قبل از نام تابع، نام کلاسی که تابع عضو آن است و به دنبال آن عملگر **::** قرار می‌گیرد.



توابع سازنده و نابود کننده

- در هر کلاس برای انجام کارهای مربوط به ایجاد شی، خصوصا مقداردهی اولیه به اعضای داده‌ای می‌توان **تابع سازنده (Constructor)** تعریف و پیاده‌سازی کرد.
- تابع سازنده تابع خاصی است که **همنام کلاس** است.
- این تابع خروجی ندارد و نوع داده خروجی (حتی void) در تعریف آن ذکر نمی‌شود.
- این تابع تنها یک بار و هنگام ایجاد یک شی **به صورت خودکار** فراخوانی می‌شود و برنامه‌نویس نمی‌تواند آن را فراخوانی کند.
- هر کلاس می‌تواند **یک یا چند** تابع سازنده داشته باشد (البته بدیهی است این توابع سازنده باید دارای امضاهای متفاوت باشند).
- اگر سازنده کلاس دارای پارامتر ورودی باشد، هنگام اعلان شی از نوع کلاس مقادیر موردنیاز باید به سازنده ارسال شود.
- در صورت عدم ایجاد تابع سازنده توسط برنامه‌نویس کامپایلر **تابع سازنده پیش‌فرضی** (بدون پارامتر) برای کلاس ایجاد خواهد کرد. این تابع به اعضای داده‌ای کلاس که از انواع داده پایه هستند، مقداری نسبت نمی‌دهد؛ ولی سازنده‌ی اعضای داده‌ای که خود از نوع کلاس دیگری هستند را فراخوانی می‌کند. در صورت ایجاد سازنده توسط برنامه‌نویس، این سازنده پیش‌فرض ایجاد نمی‌شود.
- در نسخه‌های قدیمی امکان مقداردهی اولیه اعضای داده درون کلاس وجود نداشت. اما از C++11 به بعد این امکان فراهم شده است.
- در هر کلاس برای انجام کارهای مربوط به از بین رفتن شی می‌توان **تابع نابود کننده (Destructor)** تعریف و پیاده‌سازی کرد.
- تابع نابودکننده نیز **همنام کلاس همراه با علامت ~** است.
- تابع نابودکننده خروجی و پارامتر ورودی ندارد. مشابه سازنده در تعریف نابودکننده نوع داده خروجی ذکر نمی‌شود.
- تابع نابود کننده هم تابعی است که **به صورت خودکار** در زمان از بین رفتن شی فراخوانی می‌شود.
- هر کلاس تنها **یک** تابع نابودکننده می‌تواند داشته باشد.



```
1  #include <iostream>
2  using namespace std;
3  // تعریف کلاس
4  class Circle
5  {
6  public:
7      Circle(int = 1); // تابع سازنده
8      ~Circle(); // تابع نابودکننده
9      void setRadius(int r); // تابع مقداردهی به شعاع
10     int getRadius(); // تابع دستیابی به مقدار شعاع
11     float getPerimeter(); // تابع محاسبه محیط
12     float getArea(); // تابع محاسبه مساحت
13
14 private:
15     int Radius;
16 };
17 // پیاده سازی توابع کلاس
18 Circle::Circle(int r)
19 {
20     setRadius(r);
21 }
22
23 Circle::~~Circle()
24 {
25     cout << "circle was destructed! Radius =" << Radius << endl;
26 }
27
28 void Circle::setRadius(int r)
29 {
30     if (r > 0)
31         Radius = r;
32     else
33     {
34         cout << "Radius is invalid! Set Radius = 1" << endl;
35         Radius = 1;
36     }
37 }
38
39 int Circle::getRadius()
40 {
41     return Radius;
42 }
43
44 float Circle::getPerimeter()
45 {
46     return 2 * Radius * 3.14;
47 }
48
49 float Circle::getArea()
50 {
51     return Radius * Radius * 3.14;
52 }
```



جدا کردن رابط از پیاده‌سازی

- در پیاده‌سازی انجام شده تعریف کلاس و پیاده‌سازی و همچنین تابع main در یک فایل قرار دارد.
- طبق اصل مخفی سازی اطلاعات و انتزاع داده بهتر است جزییات پیاده‌سازی را از دید استفاده‌کنندگان از کلاس مخفی کرد.
- برای این کار می‌توان تعریف کلاس را در فایل جداگانه‌ای همانم کلاس با **پسوند h**. قرار داد (header file).
- پیاده‌سازی توابع هم می‌توان در فایل جداگانه‌ای همانم کلاس با **پسوند cpp**. ذخیره کرد (source-code file). در این فایل header کلاس (**با دستور #include**) ضمیمه می‌شود.
- در کد هر برنامه‌ای که در آن از این کلاس استفاده می‌شود، نیز باید header کلاس (**با دستور #include**) ضمیمه شود. ولی می‌توان به جای فایل پیاده‌سازی شده متدهای کلاس (source-code file)، نسخه کامپایل شده آن را (object-code file) در اختیار استفاده‌کنندگان از کلاس قرار داد و به این ترتیب جزییات پیاده‌سازی را از دید استفاده‌کنندگان مخفی کرد.



فایل Circle.h

```
1  #ifndef CIRCLE_H
2  #define CIRCLE_H
3
4
5  class Circle
6  {
7      public:
8          Circle(int = 1); // تابع سازنده
9          ~Circle(); // تابع نابودکننده
10         void SetRadius(int r); // تابع مقداردهی به شعاع
11         int GetRadius(); // تابع دستیابی به مقدار شعاع
12         float GetPerimeter(); // تابع محاسبه محیط
13         float GetArea(); // تابع محاسبه مساحت
14
15         protected:
16
17         private:
18             int Radius;
19     };
20
21 #endif // CIRCLE_H
```



فایل Circle.cpp

```
1  #include<iostream>
2  #include "Circle.h" ← ضمیمه کردن فایل معرفی کلاس (class header)
3  using namespace std;
4
5  Circle::Circle(int r)
6  {
7      SetRadius(r);
8  }
9
10 Circle::~~Circle()
11 {
12     cout << "circle was destructed! Radius =" << Radius << endl;
13 }
14
15 void Circle::SetRadius(int r)
16 {
17     if (r > 0)
18         Radius = r;
19     else
20     {
21         cout << "Radius is invalid! Set Radius = 1" << endl;
22         Radius = 1;
23     }
24 }
25
26 int Circle::GetRadius()
27 {
28     return Radius;
29 }
30
31 float Circle::GetPerimeter()
32 {
33     return 2 * Radius * 3.14;
34 }
35
36 float Circle::GetArea()
37 {
38     return Radius * Radius * 3.14;
39 }
```



فایل main.cpp

← ضمیمه کردن فایل معرفی کلاس (class header)

```
1  #include <iostream>
2  #include "Circle.h"
3
4  using namespace std;
5
6  int main()
7  {
8      Circle c1(10), c2, circles[5] = {100,200,300}, *cPointer;
9      int r;
10
11     cout << "Radius of c1 is " << c1.GetRadius() << endl;
12     cout << "Perimeter of c1 is " << c1.GetPerimeter() << endl;
13     cout << "Area of c1 is " << c1.GetArea() << endl;
14     cout << "-----\n";
15
16     cout << "Radius of c1 is " << c2.GetRadius() << endl;
17     cout << "Perimeter of c2 is " << c2.GetPerimeter() << endl;
18     cout << "Area of c2 is " << c2.GetArea() << endl;
19     cout << "-----\n";
20
21     for (int i = 0; i < 5; i++)
22     {
23         cout << "Radius of circels[" << i << "] is " << circles[i].GetRadius() << endl;
24         cout << "Perimeter of circels[" << i << "] is " << circles[i].GetPerimeter() << endl;
25         cout << "Area of circels[" << i << "] is " << circles[i].GetArea() << endl;
26     }
27     cout << "-----\n";
28
29     cin >> r;
30     c1.SetRadius(r);
31     cout << "Radius of c1 is " << c1.GetRadius() << endl;
32     cout << "Perimeter of c1 is " << c1.GetPerimeter() << endl;
33     cout << "Area of c1 is " << c1.GetArea() << endl;
34     cout << "-----\n";
35
36     cPointer = &c2;
37     cout << "Radius of cPointer is " << (*cPointer).GetRadius() << endl;
38     cPointer->SetRadius(50); // (*cPointer).SetRadius(50);
39     cout << "Radius of cPointer is " << cPointer->GetRadius() << endl;
40     cout << "Radius of cPointer is " << c2.GetRadius() << endl;
41     cout << "-----\n";
42
43     cPointer = new Circle(1000);
44     cout << "Radius of cPointer is " << cPointer->GetRadius() << endl;
45     cout << "Perimeter of c2 is " << cPointer->GetPerimeter() << endl;
46     cout << "Area of c2 is " << cPointer->GetArea() << endl;
47     delete cPointer;
48     cout << "-----\n";
49     cout << "-----End of Main-----\n";
50
51     return 0;
52 }
```

```
"C:\Users\Maryam\courses\Advanced programing\SecondC..."
Radius of c1 is 10
Perimeter of c1 is 62.8
Area of c1 is 314
-----
Radius of c1 is 1
Perimeter of c2 is 6.28
Area of c2 is 3.14
-----
Radius of circels[0] is 100
Perimeter of circels[0] is 628
Area of circels[0] is 31400
Radius of circels[1] is 200
Perimeter of circels[1] is 1256
Area of circels[1] is 125600
Radius of circels[2] is 300
Perimeter of circels[2] is 1884
Area of circels[2] is 282600
Radius of circels[3] is 1
Perimeter of circels[3] is 6.28
Area of circels[3] is 3.14
Radius of circels[4] is 1
Perimeter of circels[4] is 6.28
Area of circels[4] is 3.14
-----
20
Radius of c1 is 20
Perimeter of c1 is 125.6
Area of c1 is 1256
-----
Radius of cPointer is 1
Radius of cPointer is 50
Radius of cPointer is 50
-----
Radius of cPointer is 1000
Perimeter of c2 is 6280
Area of c2 is 3.14e+06
circle was destructed! Radius =1000
-----
-----End of Main-----
circle was destructed! Radius =1
circle was destructed! Radius =1
circle was destructed! Radius =300
circle was destructed! Radius =200
circle was destructed! Radius =100
circle was destructed! Radius =50
circle was destructed! Radius =20
Process returned 0 (0x0)   execution time : 5.908 s
Press any key to continue.
```